

Propondría **Evolución Diferencial (DE)** como opción principal.

La razón es que el problema trabaja con **variables continuas reales**: posiciones, orientación parametrizada, factores de velocidad y posición del raíl. Además, tras el rediseño del alumno, gran parte de la factibilidad se garantiza antes de optimizar, y el nivel 2 queda reducido a pocas variables continuas: velocidades y posición del raíl. Ahí DE encaja muy bien porque genera nuevas soluciones mediante diferencias entre individuos reales y adapta los desplazamientos a la diversidad de la población.

Frente a **GA**, DE sería más sencillo y natural: no necesita diseñar selección, cruce y mutación complejos para variables heterogéneas. Frente a **PSO**, evitaría en parte la convergencia prematura hacia velocidades máximas, porque DE mantiene una población diversa y perturba usando diferencias entre soluciones, no solo atracción hacia mejores locales/globales. El material del Tema 3 también señala que PSO y DE suelen ser más simples computacionalmente que GA para prototipos rápidos, y que DE trabaja directamente con codificación real.

Lo plantearía como **DE/rand/1/bin con reparación/proyección de restricciones**:

$x = (p_A, p_B, s_A, s_B, s_rail, p_rail)$

manteniendo orientaciones fijas como en la versión revisada, o incluyéndolas después si se quiere ampliar el problema. La evaluación debería combinar:

$T_max = \max(t_A, t_B)$

con penalización por sincronización:

$P_sync(|t_A - t_B|)$

y, si se vuelve a optimizar el punto espacial, penalizaciones por separación, alineación, manipulabilidad y alcance. El propio trabajo ya usa una penalización continua para la sincronización, evitando descartar soluciones y manteniendo una guía útil para la búsqueda.

Como alternativa avanzada, si se quiere explotar la formulación original de dos tiempos separados, propondría **NSGA-II** para obtener el frente de Pareto entre t_A y t_B ; pero para la entrega del Tema 3 y para comparar claramente con el Temple Simulado, mi recomendación sería **DE monoobjetivo penalizado**.

El trabajo tiene una base bastante sólida y, de hecho, presenta una formulación bastante madura para un problema de optimización robótica aplicado. Precisamente por eso, los principales puntos de mejora no están tanto en la “idea”, sino en la **formalización experimental, el tratamiento de restricciones, la modelización temporal y la validación científica.**

Los dividiría en varios bloques.

1. Puntos débiles de la formulación del problema

1.1. La función objetivo todavía mezcla objetivos heterogéneos

Uno de los principales problemas es que el alumno convierte muchos criterios distintos en una única suma ponderada:

- tiempo máximo,
- sincronización,
- singularidades,
- manipulabilidad,
- restricciones geométricas,
- separaciones,
- etc.

Eso introduce varios riesgos:

- sensibilidad extrema a los pesos;
- dificultad para interpretar resultados;
- soluciones artificiales por compensación entre términos;
- tuning manual poco justificable.

Por ejemplo, una pequeña mejora temporal puede compensar una penalización geométrica importante dependiendo de escalas numéricas.

Mejora recomendada

Plantear explícitamente el problema como:

- **multiobjetivo (NSGA-II / MOEA/D);**
 - o jerárquico:
 1. factibilidad;
 2. sincronización;
 3. minimización temporal.

En particular, el problema original parece naturalmente biobjetivo:

$$\min_{f_0}(t_A, t_B) \min(t_A, t_B)$$

con restricción:

$$|t_A - t_B| \leq \Delta t$$

y eso encaja muchísimo mejor con algoritmos evolutivos multiobjetivo.

2. Tratamiento débil de restricciones

2.1. Penalización excesivamente dependiente de parámetros

El alumno utiliza penalizaciones continuas, lo cual es correcto y mucho mejor que descartar soluciones. Sin embargo:

- los coeficientes de penalización parecen elegidos heurísticamente;
- no hay análisis de sensibilidad;
- no se estudia qué ocurre cuando cambian escalas.

Eso puede provocar:

- búsqueda atrapada en regiones inviables;
 - o lo contrario: sobrepenalización que reduce diversidad.
-

2.2. Falta una estrategia formal de manejo de restricciones

Actualmente la propuesta se basa esencialmente en:

- generar soluciones factibles;
- penalizar violaciones.

Pero en optimización evolutiva moderna suelen compararse varias estrategias:

Técnicas más avanzadas

- repair operators;
- feasibility rules de Deb;
- adaptive penalties;
- ϵ -constrained methods;

- stochastic ranking;
 - decoders factibles.
-

2.3. La factibilidad cinemática es demasiado simplificada

El modelo usa:

- workspace;
- IK;
- singularidad;
- manipulabilidad.

Pero falta aclarar:

- si existen múltiples soluciones IK;
- cómo se elige la solución articular;
- continuidad temporal entre configuraciones;
- evitación de trayectorias inviables.

En robótica esto es muy importante porque:

una pose puede ser alcanzable de manera aislada, pero no mediante una trayectoria físicamente válida.

3. Debilidad importante: modelo temporal simplificado

Este probablemente es el punto técnico más mejorable.

Actualmente el tiempo parece aproximarse mediante:

- velocidades máximas,
- distancia,
- escalado lineal.

Pero el tiempo real depende de:

- aceleraciones,
- jerk,
- límites articulares,
- sincronización del planificador,
- perfiles trapezoidales/S-curve,

- planificación del raíl.
-

Mejora importante

Integrar:

- MoveIt!,
- OMPL,
- o tiempos reales del controlador.

Aunque sea offline.

Por ejemplo:

$$t = \max_i [f_0] \Delta q_i \quad i_{\max} t = i_{\max} q \cdot i_{\max} \Delta q_i$$

ya sería mejor aproximación que distancia euclídea pura.

4. La representación genética puede mejorarse

La codificación actual es razonable, pero tiene debilidades.

4.1. Problema con orientaciones y cuaterniones

Si usa cuaterniones directamente:

- DE puede romper la normalización;
- aparecen orientaciones inválidas.

Mejora

- renormalizar tras mutación;
 - usar exponential maps;
 - usar representación axis-angle;
 - o restringir orientación parcialmente.
-

4.2. Variables con escalas distintas

El vector mezcla:

- posiciones (metros),
- velocidades (0–1),
- cuaterniones,
- rail position.

Eso puede perjudicar mucho a DE porque:

$xr^2 - xr^3$

queda dominado por variables de mayor escala.

Mejora

Normalización explícita:

$x_i \in [0, 1]$

para todas las dimensiones antes de aplicar DE.

5. Falta explotación de estructura geométrica del problema

El trabajo formula bien el problema, pero todavía lo trata como una caja negra.

Sin embargo, el problema tiene mucha estructura:

- geometría;
- simetrías;
- redundancia;
- regiones inviables conocidas;
- separación fija entre efectores.

Mejora interesante

Usar:

- inicialización inteligente;
- operadores geométricos;
- sampling guiado;
- DE híbrido con heurísticas.

Por ejemplo:

1. generar primero puntos factibles de cooperación;
2. luego optimizar velocidades;
3. luego refinamiento local.

Eso probablemente mejoraría muchísimo convergencia.

6. Falta un estudio experimental fuerte

Este es probablemente el punto más crítico desde perspectiva académica.

Actualmente faltaría:

- número suficiente de ejecuciones;
 - desviaciones típicas;
 - boxplots;
 - contrastes estadísticos;
 - análisis de robustez;
 - comparación con baselines.
-

Imprescindible

Para que el trabajo tenga calidad científica:

Repeticiones independientes

Al menos:

3030

semillas distintas.

Comparación contra:

- búsqueda aleatoria;
- hill climbing;
- simulated annealing;
- PSO;
- GA;
- DE.

Estadística

- Kruskal-Wallis;
- Mann-Whitney;
- Wilcoxon;
- tamaño del efecto.

7. No se justifica suficientemente por qué Temple Simulado

El trabajo anterior usa Temple Simulado, pero:

- no parece alinearse especialmente bien con la estructura continua del problema;
- no aprovecha poblaciones;
- no explota paralelismo.

Argumento fuerte a introducir

Explicar por qué DE es mejor aquí:

Porque el problema es:

- continuo,
- multimodal,
- altamente restringido,
- con acoplamiento entre variables.

Y DE:

- trabaja naturalmente en continuo;
- mantiene diversidad;
- requiere pocos hiperparámetros;
- es robusto en espacios reales;
- explora mejor correlaciones geométricas.

8. La propuesta puede evolucionar hacia un trabajo muy potente

El alumno tiene realmente una base bastante prometedora.

El salto de calidad estaría en:

Línea A — Multiobjetivo real

Aplicar:

- NSGA-II,
- NSGA-III,
- MOEA/D.

y obtener:

- frente de Pareto de sincronización vs tiempo.
-

Línea B — Híbrido evolutivo + planificación

Combinar:

- DE para poses;
 - MoveIt para trayectorias;
 - evaluación temporal real.
-

Línea C — Optimización robusta

Introducir incertidumbre:

- ruido en sensores,
- retrasos,
- perturbaciones.

Y optimizar:

$E[T]E[T]$

o worst-case.

9. Qué destacaría positivamente del alumno

También señalaría fortalezas claras:

Muy positivo

- buena formalización matemática;
- comprensión razonable del problema robótico;
- buen uso de penalizaciones suaves;
- simplificación inteligente del espacio de búsqueda;
- separación conceptual entre geometría y tiempos;
- intuición correcta sobre sincronización;
- identificación de singularidades/manipulabilidad;
- reducción progresiva de dimensionalidad.

Eso no suele aparecer en trabajos iniciales.

10. Recomendación concreta que yo le daría

Le diría algo así:

“La formulación del problema es buena y suficientemente rica como para justificar el uso de metaheurísticas evolutivas modernas. El siguiente salto de calidad no está en añadir más restricciones, sino en mejorar el rigor experimental, el manejo formal de restricciones, la representación de variables y la validación estadística. El problema parece especialmente adecuado para Evolución Diferencial o técnicas multiobjetivo tipo NSGA-II.”

Le propondría estos cambios concretos para que DE encaje mejor y la entrega sea más sólida:

1. Normalizar todas las variables a $[0,1]$.

DE trabaja con diferencias entre vectores; si mezcla metros, cuaterniones y velocidades, unas variables dominan a otras. Optimizaría internamente en $[0,1]$ y después decodificaría a unidades físicas.

2. Mantener una arquitectura en dos niveles.

Primero generar puntos espaciales factibles de rendezvous; después aplicar DE a las variables temporales:

$$x=(s_A, s_B, s_{rail}, p_{rail})$$

Esto reduce ruido, evita muchas soluciones inviables y permite comparar directamente con el Temple Simulado.

3. Usar DE/rand/1/bin como versión base.

Configuración inicial razonable:

$$NP=10d \text{ a } 20d, F=0.5-0.8, CR=0.7-0.9$$

y 30 ejecuciones independientes.

4. Añadir una versión DE adaptativa.

Después de la versión básica, probaría jDE o una adaptación sencilla donde cada individuo tenga su propio FiFi y CRi. Así evita fijar manualmente parámetros sensibles.

5. Separar claramente objetivo y restricciones.

No mezclar todo en una única suma sin justificar. Mejor usar reglas de factibilidad:

1. una solución factible domina a una no factible;
2. entre dos factibles, gana menor T_{max} ;
3. entre dos no factibles, gana menor violación total.

Esto es más limpio que ajustar pesos arbitrarios.

6. Penalizar explícitamente el sesgo hacia velocidad máxima.

El alumno detecta que las soluciones tienden a $s=1$. Añadiría un término de suavidad:

$$P_{speed} = \lambda s [(s_A - s_{Aref})^2 + (s_B - s_{Bref})^2 + (s_{rail} - s_{railref})^2]$$

o una restricción de aceleración/jerk si puede modelarla.

7. Introducir reparación de soluciones.

Después de mutación y cruce:

- recortar velocidades a $(0,1](0,1]$;
- proyectar prailprail al intervalo alcanzable;
- renormalizar cuaterniones si se optimizan orientaciones;
- descartar o reparar poses fuera del workspace.

8. Comparar contra baselines.

Mínimo:

- Temple Simulado del trabajo anterior;
 - búsqueda aleatoria;
 - DE básico;
 - DE adaptativo;
 - opcionalmente PSO.

9. Medir más que solo $T_{\max}^{f_0} T_{\max}$.

Reportaría:

- $T_{\max}^{f_0} T_{\max}$;
- $|t_A - t_B| |t_A - t_B|$;
- porcentaje de factibilidad;
- número de evaluaciones hasta convergencia;
- tiempo de CPU;
- distribución de $s_A, s_B, s_{\text{rails}A}, s_{\text{rails}B}, s_{\text{rail}}$;
- violación media de restricciones.

10. Usar análisis estadístico.

Con 30 semillas:

- boxplots;
- media, mediana, desviación típica;
- intervalo de confianza;
- Kruskal-Wallis si compara varios algoritmos;
- Mann-Whitney/Wilcoxon por pares;
- tamaño del efecto.

La recomendación central sería: **empezar con un DE monoobjetivo restringido y normalizado sobre el nivel temporal**, y luego ampliar a un **NSGA-II** si quiere estudiar el compromiso entre t_{AtA} , t_{BtB} , suavidad y sincronización.